

ViRGo — Guide

Onboarding guide. Read this once, top to bottom, and you can run and extend the project on your own.

1. What is this project?

ViRGo (*Virtual Role-Graph Embedding for Structural Identity*) is a research project aiming at a published paper. It **extends** an existing method called **Identity2Vec (I2V)** by Oluigbo et al.

Plain-English background.

- A **graph** = dots joined by lines. Dots are **nodes** (e.g. web pages, papers, people); lines are **edges** (a link, a citation, a friendship).
- Two nodes can share a **role** even if far apart — both *hubs* (many links) or both *bridges* (connect two groups). This role is a node's **structural identity**.
- An **embedding** = a list of 64 numbers per node (its "fingerprint"). Nodes with similar roles should get similar fingerprints.
- **I2V** builds these fingerprints with a *guided random walk* + a word model (Word2Vec). ViRGo studies a better way to feed structure into them.

2. Goal, contributions, methodology

Research question: which **virtual graph** (a graph that links nodes by *structural similarity*, not real edges) is best for a given dataset/task, and does a modern **GNN** over it beat I2V's walk method?

#	Contribution	Status
1	Virtual-graph study — build virtual graphs from I2V's Poisson/KL similarity; test on node classification, link prediction, anomaly detection; compare vs simpler graphs.	main work
Tech	GNN encoder (GraphSAGE/GIN/GAT) over the virtual graph, replacing walk+Word2Vec.	Phase 3
2	Embeddings as graph summaries for LLMs.	stretch, do not start yet

Project phases:

1. **Phase 1 — reproducibility (match the I2V paper).** ✓ **done** — cached I2V + cross-model baseline comparison (baselines used as-is, **not fine-tuned**); within ± 0.05 of the paper.
2. **Phase 2 — virtual-graph creation** ← next — top-K Poisson/KL Ψ graph + degree-only / centrality-only comparison graphs.
3. **Phase 3 — modern GNN encoder** — GraphSAGE / GIN / GAT over the virtual graph, replacing walk + Skipgram (technical contribution; design + compare variants).
4. **Phase 4 — downstream tasks** — node classification, link prediction, anomaly detection (new); virtual-graph ablation (which graph best per data/task).
5. **Phase 5 — LLM context-window issue** — compact structural embeddings as a large-graph summary (stretch; not yet).

Pipeline (the method): graph → structural signal (degree + eigenvector centrality, cached) → KL-divergence λ → Poisson Ψ similarity → [virtual graph + GNN = future] → embedding → evaluation. Out of scope: hyperbolic/non-Euclidean space (a second paper).

3. Repository map (what each file does)

Core algorithm + run scripts live in the project root. The reusable CLI framework lives in `scripts/`. Everything else is data, docs, or results.

File / folder	Purpose
<code>identity2vec.py</code>	CORE baseline (frozen, never edit). The I2V <code>Graph</code> class + guided walk: uses node degree & eigenvector centrality → KL → Poisson Ψ to pick the next node.
<code>identity2vec_cached.py</code>	Same algorithm but caches the structural signals → identical output, ~200× faster . (Deliverable #1.)
<code>train.py</code>	The run file : read graph → make walks → Word2Vec (Skipgram) → save <code>.emb</code> . Flag <code>--cached</code> uses the fast path.
<code>make_labels.py</code>	Downloads & builds node labels (cora/citeseer from LINQS; webkb from the author's repo) and verifies they match our graph.
<code>prepare_linkpred.py</code>	Builds the link-prediction edge split (70/30) + fake "negative" pairs, leakage-free.
<code>eval_nodeclass.py</code>	Scores node classification → micro / macro / weighted F1.
<code>eval_linkpred.py</code>	Scores link prediction → AUC (cosine = headline; Hadamard + logreg = second column).

File / folder	Purpose
plot_emb.py	Draws an embedding as a 2-D picture (optional).
scripts/ benchmark_config.py	Single source of truth: dataset registry, hyperparameters, seed, split fractions.
scripts/runner.py	Glue: <code>embed()</code> , <code>run_linkpred()</code> , <code>run_nodeclass()</code> , plus the 3-seed <code>run_*_repeated()</code> .
scripts/results_io.py	Saves a run to <code>results/NNN.<date>.<dataset>.<task>.csv</code> .
scripts/utils.py	Helpers: set seed, load embedding, next run id.
scripts/main.py	Terminal CLI (alternative to the notebook).
notebooks/ reproduce_i2v.ipynb	★ Primary entry point — click-through reproduction (see §5).
input/	Original graphs (<code>.edgelist</code>). Never edit.
output/	Trained embeddings (<code>.emb</code>).
labels/	Node categories for classification.
splits/	Edge splits for link prediction.
results/	Score sheets (<code>.csv</code>) + plots (<code>.png</code>).
docs/	The paper PDF, proposal, <code>notes.md</code> (lab notebook), this guide .
CLAUDE.md	The project's governing rules (read it).

4. Setup (one time)

1. The project uses the **conda environment i2v** (Python 3.12). Activate it: `conda activate i2v` — it already has numpy 1.26.4, networkx, gensim 4.3.3, scipy 1.12.0, scikit-learn, pandas, jupyter.
2. Open the notebook in VS Code (or `jupyter lab`) and pick the kernel "**Python (i2v)**".
3. **Internet** is needed the first time you build cora/citeseer labels (a download).
4. A harmless `libtinfo.so` warning may appear in the terminal — ignore it.

5. How to run (the notebook = single entry point)

Open `notebooks/reproduce_i2v.ipynb`, run cells **top to bottom (Shift+Enter)**. You edit **one cell** (Step 1).

- **Step 0** — set up (imports, paths).
- **Step 1** — set `DATASET` and the seed list (e.g. `[42, 43, 44]`). ← the only cell you edit; every cell below follows it.

- **Step 2 — node classification:** train (if missing) one I2V embedding per seed → logistic regression → micro/macro/weighted F1.
- **Step 3 — link prediction:** per seed make the 70/30 split → retrain on the 70% graph → Hadamard + logreg → AUC.
- **Step 4 — results + summary:** mean $\pm$ std over the seeds, saved to `results/`.
- **Step 5 — cross-model benchmark:** I2V vs DeepWalk / node2vec / struc2vec on the benchmark datasets → tables in `results/`.

CLI alternative (terminal):

```
python scripts/main.py --list # show datasets
python scripts/main.py --task nodeclass --dataset cora
python scripts/main.py --task linkpred --dataset cora --retrain
```

6. Datasets

Registered in `scripts/benchmark_config.py` (DATASETS). Labelled & verified: **cora** (7 classes), **enzymes**, **webkb_wisc** (5 classes). The author **citeseer** graph has no aligned labels — use `citeseer_linqs` for node classification (author `citeseer` = link-pred only). **politics** ships no labels (link-pred only). The cross-model benchmark (`BENCH_DATASETS`) sweeps **cora** • **citeseer_linqs** • **enzymes** • **webkb_wisc**.

7. Evaluation — splits, settings, metrics

The most important idea: the two tasks split differently.

Node classification — transductive. Build the embedding on the **whole** graph once, then split the **nodes** 70/30 (`train_frac=0.7`, stratified, keeps class ratios), train a one-vs-rest logistic-regression classifier (`OneVsRestClassifier(LogisticRegression(solver="lbfgs", max_iter=300))`) on 70%, score the held-out 30%. The test *labels* are hidden → no leakage. **Metrics: micro / macro / weighted F1.**

Link prediction — inductive, leakage-free. Split the **edges** 70/30. A *spanning tree* stays in train so the graph stays connected; equal numbers of fake "negative" (non-edge) pairs are added. **Retrain a fresh embedding on the 70% train graph only** (so test edges are never seen). Score AUC two ways: **unsupervised cosine similarity** of the two node vectors — the **headline** metric, paper-faithful, no classifier — and a supervised **Hadamard-feature logistic regression** (node2vec protocol), kept as a second column. **Metric: AUC (cosine headline).**

Fixed settings: `seed = 42` (benchmark sweeps seeds 42, 43, 44); embedding dimensions = 64, num_walks = 10, window = 10, epochs = 1; walk_length = **40** everywhere (train.py, benchmark_config, notebook) — the repo default (the paper's 80 is $\approx 1.87\times$ slower, kept as a recorded deviation, see docs/notes.md). Classifier = one-vs-rest logistic regression (L-BFGS, 300 iters, L2 — the paper's protocol). No separate validation set (we don't tune).

Caveat (research rigor): the pipeline reports **mean $\pm$ std over 3 seeds** (42/43/44) with macro-F1 included. Small datasets stay noisy — widen to **≥ 5 seeds** for the final paper.

8. Results & outputs

Where	What
<code>output/{ds}/{ds}_nc_orig_s{seed}.emb</code> (I2V) · <code>{model}_{ds}_nc_orig_s{seed}.emb</code> (baselines)	full-graph embedding per model/seed (node classification)
<code>output/{ds}/..._lp_orig_s{seed}.emb</code>	train-only embedding per model/seed (link prediction)
<code>splits/{ds}_train.edgelist,</code> <code>_train_neg.txt,</code> <code>_test_pos.txt,</code> <code>_test_neg.txt</code>	link-pred split
<code>results/NNN.DD.MM.{ds}.{task}.csv</code>	one run: a #META JSON header (all settings) + metric rows; auto-numbered
<code>results/table*.csv</code>	cross-model benchmark tables (Step 5); LP headline = cosine AUC, logreg AUC kept in <code>benchmark_per_seed.csv</code>

Reproduced so far: Cora I2V lands in the paper's range on both tasks — the live numbers are whatever the latest 3-seed run wrote to `results/` (report **mean $\pm$ std**, not a single value). webkb_wisc node-class F1 $\approx$ random (structure $\neq$ content topics — see §10); webkb is **not** a paper benchmark. Cache fix: **207 $\times$ faster, byte-identical** (verified, Deliverable #1). NOTE: all `.emb` were just cleared for the walk-length-40 revert — re-run the notebook to regenerate before quoting any metric.

9. Reproducibility rules (non-negotiable)

- **seed = 42 everywhere** (split, init, sampling). Keep `--workers 1` (gensim is otherwise non-deterministic).
- **Never modify anything in `input/`** — write derived files alongside or into `output/`.
- A result counts as "**reproduced**" **only within ± 0.05** of the paper.

- **Log every run and decision in** `docs/notes.md` (the lab notebook).

10. Coding rules & where to change things

Coding style (match I2V): `argparse CLI`; `build_graph()` / `learn_embeddings()` / `main(args)` `shape`; **fewest functions**, each short with a one-line docstring; self-explanatory file names; models expose `train(epochs)` not `fit()`; add no dependency without need.

✓ Safe modification zones

- Pick dataset/task → notebook **Step 1** (`DATASET`), then run the node-class or link-pred cells.
- Tune hyperparameters → notebook **Step 1** vars, or `scripts/benchmark_config.py` (`I2V_PARAMS`) for the CLI.
- Add a dataset → drop the `.edgelist` in `input/`, add an entry to `DATASETS`, add labels in `labels/`.
- Add a task → new `eval_<task>.py` + register it in `scripts/runner.py` (`TASKS`).

Do NOT touch

- `identity2vec.py` (frozen baseline) and don't move it or `train.py`.
- Files in `input/`. The `seed`. The location of `CLAUDE.md` (root).

11. Known issues / gotchas

- **Notebook clobber:** if the `.ipynb` is open in an editor while a script edits it, the editor can overwrite changes. Close it before bulk edits; reload from disk after.
- **citeseer labels** may be rejected by the overlap safety-check (derived `edgelist`) → node classification can stop. Link prediction still works.
- **webkb structure ≠ content:** I2V learns *structural roles*; webkb classes are *content topics* and the graph is heterophilous, so node-class F1 ≈ random. Expected, not a bug. Use `webkb_wisc` for *link prediction*.
- **Single-split variance** on small graphs (§7 caveat).
- The original I2V recomputes centrality inside the walk loop (very slow) — always use `--cached`.

12. Deliverables & how to continue

Deliverable	State	Next action
1. Cached I2V (identical + faster)		—

Deliverable	State	Next action
	✓ done · Phase 1	
—. I2V reproduction + cross-model baseline comparison	✓ done · Phase 1	baselines used as-is, not fine-tuned
2. <code>virtual_graph.py</code> (top-K Poisson/KL builder)	● next · Phase 2	top-K Ψ graph + degree-only / centrality-only comparison graphs
3. GNN encoder (GraphSAGE/GIN/GAT) over the virtual graph	• Phase 3	design + compare architecture variants (add torch + torch-geometric)
4. Anomaly detection eval (AUC/AP)	• Phase 4	with node-class + link-pred; + virtual-graph ablation
5. Benchmark table + ablation (vs I2V, over graph & K)	• Phase 4	extend <code>results/</code>
6. Paper draft	•	—

Where to start as the new intern: (1) run the notebook on **cora** end-to-end and confirm the 3-seed mean $\pm$ std in `results/` sits in the paper's range; (2) read `identity2vec.py` and `train.py` to see how walks become embeddings; (3) skim `docs/notes.md` for history; (4) then start **Phase 2** — build `virtual_graph.py` (top-K Ψ + degree/centrality comparison graphs); **Phase 3** (the GNN encoder, GraphSAGE / GIN / GAT) follows.

13. Mini-glossary

Node/edge — dot / line. **Embedding** — 64-number fingerprint per node.

Structural identity — a node's role (hub, bridge), independent of position.

Transductive — embed the whole graph, hide only labels. **Inductive** — retrain on a sub-graph so test items are unseen. **Weighted F1** — accuracy-like score that accounts for class sizes. **AUC** — probability the model ranks a real edge above a fake one. **Leakage** — letting test information into training (forbidden).